Distributed-Memory Parallelisation of a Barnes–Hut N-Body Simulation with MPI and OpenMP

IT4130E Parallel and Distributed Programming

Chu Xuan Minh (20235527) Nguyen Binh Anh (20235470)

Doan Phuong Khang (20235510) Nguyen Tien Phat (20235544)

June 24, 2026

Abstract

We study the gravitational N-body problem, in which the trajectories of N mutually attracting masses must be advanced through time. Evaluating all pairwise forces directly costs $\mathcal{O}(N^2)$ per step; the Barnes–Hut tree approximation reduces this to $\mathcal{O}(N \log N)$ by grouping distant masses. We implement the two-dimensional method as a sequential reference and parallelise it for distributed memory with the Message Passing Interface (MPI), adding a second, shared-memory level of parallelism with OpenMP. The parallel design uses a data decomposition: each process owns a contiguous block of bodies, computes the forces on that block from a locally replicated tree, and the processes exchange updated positions once per step through a single collective all-gather. We deploy the program on a real four-machine cluster — four laptops, one two-core virtual machine each, joined over a phone Wi-Fi hotspot — giving eight cores in total. Correctness holds end to end: the distributed program reproduces the sequential reference *bit for bit* on all eight cores, and total energy is conserved to within 0.6 %. The performance result is shaped entirely by the network. Because the algorithm performs two blocking all-gathers per time step, and the hotspot link has millisecond-scale latency, *communication accounts for 98–99% of the wall-clock time* at every problem size, and the end-to-end speed-up is below one — adding networked processes makes the program slower, not faster. Separating the two costs tells the real story: the *computation* alone scales well, falling about fivefold from one to eight cores, while the communication term dominates the total. We report this communication-bound behaviour honestly as the central finding, explain it with a simple latency cost model, and identify the algorithmic change — communicating less often — that a high-latency interconnect would require.

Contents

1 Introduction

The gravitational N-body problem asks how N point masses move under their mutual Newtonian attraction. It is a canonical high-performance computing workload: it underlies simulations of star clusters, galaxies and cosmological structure formation [7], and its computational cost grows quickly with the number of bodies. A direct evaluation of the force on every body sums the contributions of all the others, costing $\mathcal{O}(N^2)$ operations per time step, which becomes prohibitive well before the problem is scientifically interesting.

Barnes and Hut [1] observed that a distant cluster of masses may be replaced, to controllable accuracy, by a single equivalent mass at its centre of mass. Organising the bodies in a spatial tree and applying this approximation lowers the per-step cost to $\mathcal{O}(N \log N)$. The tree, however, makes the computation irregular: the work to evaluate the force on a body depends on how the other bodies are distributed in space, which complicates an even division of work across processors.

This project pursues three objectives. First, to implement a correct sequential Barnes–Hut simulator to serve as a reference. Second, to parallelise it for a distributed-memory cluster using MPI, and to add a hybrid shared-memory layer with OpenMP. Third, to deploy it on a genuine multi-machine cluster and measure the resulting performance — correctness, running time, load balance and speed-up — explaining the results in terms of the decomposition, mapping and communication choices. A central finding emerges from the third objective: on a commodity wireless network the cost of communication overwhelms the cost of computation, so the honest performance story is one of a communication-bound program, and much of our analysis is devoted to separating the two costs and explaining why. The remainder of the report is organised as follows. Section 2 reviews the algorithm and the parallel-design concepts we use. Section 3 states the sequential method. Section 4 develops the parallelisation strategy. Section 5 describes the cluster and the experimental setup, and Section 6 reports the measurements. Section 7 discusses limitations and Section 8 concludes.

2 Background

The Barnes–Hut approximation. The plane is recursively divided into quadrants, forming a quadtree whose leaves hold at most one body. Every internal node stores the total mass and centre of mass of the bodies beneath it. To compute the force on a body, the tree is traversed from the root: if a node is sufficiently far away — its width s and the distance d to its centre of mass satisfy $s/d < \theta$ for an opening parameter θ — the whole subtree is treated as one mass; otherwise the traversal descends into its children. Smaller θ gives higher accuracy at greater cost; $\theta = 0.5$ is the standard choice and keeps the force error near one percent [1].

Designing a parallel algorithm. Following the standard methodology [3, 4], a parallel design is described by four decisions. *Decomposition* divides the computation into tasks; common strategies are data decomposition (partitioning the data each task works on), recursive decomposition (the divide-and-conquer structure of the problem), and exploratory or speculative

decomposition for search problems. The *granularity* of a decomposition is the amount of work per task relative to the communication it requires. *Mapping* assigns tasks to processors — for an array-shaped domain, typically a one-dimensional block, a two-dimensional block, or a cyclic distribution. *Communication* specifies how processes exchange data: point-to-point or collective, blocking or non-blocking, and with what logical topology. Two laws bound the outcome: a parallel program is limited by its sequential fraction (Amdahl's law [5]), and its *efficiency* — speed-up divided by processor count — falls as the overhead of communication and idle time grows. The message-passing model itself, in which processes with private memories cooperate by explicitly sending data [2], is realised here through MPI.

3 The sequential algorithm

Physical model. Each body i has position $\vec{r}_i$, velocity $\vec{v}_i$ and mass m_i. The acceleration on body i is the softened gravitational sum

$$\vec{a}_i \;=\; G\sum_{j\neq i} m_j \,\frac{\vec{r}_j-\vec{r}_i}{\left(\|\vec{r}_j-\vec{r}_i\|^2+\varepsilon^2\right)^{3/2}}, \tag{1}$$

where G is the gravitational constant (set to one in scaled units) and ε is a softening length that removes the singularity when two bodies approach each other. We adopt standard N-body units in which the total mass is one, and set ε comparable to the mean spacing of the bodies so that close encounters remain resolvable at the chosen time step; this keeps the total energy bounded, as verified in Section 6.

Time integration. Velocities and positions are advanced with the semi-implicit (Euler–Cromer) scheme,

$$\vec{v}_i^{\,n+1}=\vec{v}_i^{\,n}+\vec{a}_i^{\,n}\,\Delta t, \qquad \vec{r}_i^{\,n+1}=\vec{r}_i^{\,n}+\vec{v}_i^{\,n+1}\,\Delta t,$$

which updates the velocity from the current acceleration and then the position from the new velocity. This first-order method is symplectic, so the energy error stays bounded over long runs rather than growing without limit, which is the property a demonstration of physical correctness requires.

Initial conditions. The bodies are sampled from a Plummer model [6], a classical equilibrium model of a spherical star cluster, with tangential velocities near the local circular speed so the cluster neither collapses nor disperses over the interval simulated.

Cost. Each step builds the tree, accumulates the centres of mass, evaluates the force on every body by a tree traversal of average depth $\mathcal{O}(\log N)$, and integrates. The force evaluation dominates, giving $\mathcal{O}(N\log N)$ work per step. Algorithm 1 summarises one step.

Algorithm 1 One Barnes–Hut time step (sequential).

```
compute the bounding square of all body positions
build the quadtree over the bodies
accumulate mass and centre of mass at every node (post-order)
for each body i do
    a_i ← traverse the tree from the root with opening parameter θ
end for
for each body i do
    v_i += a_i Δt;   r_i += v_i Δt
end for
```

4 Parallelisation strategy

This section answers, in turn, the design questions of Section 2: the level and technique of decomposition, the mapping of work to processes, the communication strategy and topology, and load balance.

Level of parallelism and decomposition technique. The parallelism is at the level of *data*: the set of bodies is the data domain, and every process applies the identical operation — a force evaluation followed by an integration — to a different part of it. The decomposition technique is therefore a *data decomposition* under the owner-computes rule, where the process that owns a body is responsible for computing its force and advancing it. The tree construction underneath is a *recursive* divide-and-conquer computation, but in this design it is *replicated* on every process rather than itself divided; the combination of a data decomposition of the force work with a recursive (replicated) tree build is, in the standard taxonomy, a hybrid decomposition dominated by its data-parallel component. Replicating the tree is a deliberate trade: it costs every process an $\mathcal{O}(N)$ build and $\mathcal{O}(N)$ storage but removes all communication from the force traversal, because each process already holds the complete tree and needs no remote data to evaluate any force.

Mapping. With P processes and N bodies, let $b = \lfloor N/P \rfloor$. Process r owns the contiguous index block from rb up to $(r{+}1)b$, the remainder being spread one body per process. This is a *one-dimensional block* mapping of the body array. A two-dimensional $\sqrt{P} \times \sqrt{P}$ block mapping, appropriate for a matrix-shaped domain, does not apply here because the work domain is a one-dimensional list of bodies rather than a grid; we return to this choice when analysing load balance.

Communication strategy and topology. The processes follow a single-program, multiple-data pattern: every process runs the same code on its own block and there is no distinguished master that farms out work, so the scheme is symmetric rather than master–slave. Communication occurs at two points. After initialisation, the starting state is sent from one process to all others by a *broadcast*. Then, once per time step, every process must see the updated positions of all bodies in order to rebuild its tree for the next step; this is achieved by a single *collective all-gather*, in which each process contributes its block of positions and receives the concatenation of all blocks. All communication is *blocking*: a process waits for the exchange to complete before continuing. We chose blocking, collective communication over non-blocking point-to-point exchange because the per-step volume is small — two coordinates per body — and a single collective is both simpler to reason about and well optimised inside the MPI library. The all-gather imposes no explicit logical topology such as a ring or hypercube; it is an all-to-all exchange that the library typically realises internally by a recursive-doubling (hypercube) or ring algorithm. A final max-reduction collects timing information at the end of a run. Algorithm 2 states the parallel step.

Algorithm 2 One Barnes–Hut time step on process r of P (MPI; the hybrid program threads the marked loop with OpenMP).

(once) broadcast the initial positions, velocities and masses to all processes
$lo, hi \leftarrow$ bounds of the block owned by process r ▷ 1-D block mapping
for each time step **do**
    build the full quadtree from the positions held locally ▷ replicated
    accumulate mass and centre of mass at every node
    for $i \leftarrow lo$ **to** $hi - 1$ **do** ▷ *parallel-for* in the hybrid version
        $\vec{a}_i \leftarrow$ traverse the tree with opening parameter θ
    end for
    for $i \leftarrow lo$ **to** $hi - 1$ **do**
        $\vec{v}_i \mathrel{+}= \vec{a}_i \, \Delta t; \quad \vec{r}_i \mathrel{+}= \vec{v}_i \, \Delta t$
    end for
    all-gather the updated positions so every process can rebuild next step
end for

Load balance. The blocks are equal in *size*, but the work to evaluate a force is not equal across bodies: a body in a dense region opens more tree nodes than one in a sparse region. Equal counts therefore do not guarantee equal work. Whether the imbalance matters depends on how bodies are ordered in the array. When the ordering is unrelated to position — as in our generated input — each block is a random spatial sample and the loads are nearly equal. When bodies are ordered by position, each block becomes a compact spatial region of very different density, and the imbalance grows. We quantify both cases in Section 6 using the idle-time criterion and discuss the remedy.

Cost model. Per step, each process performs an $\mathcal{O}(N)$ tree build, an $\mathcal{O}((N/P)\log N)$ force evaluation over its block, and one all-gather of $\mathcal{O}(N)$ coordinates. The force term shrinks with P, so on an ideal machine the computation would scale well. The communication term, however, has two parts: a *bandwidth* cost proportional to the data volume and, more importantly here, a *latency* cost paid once per collective regardless of volume. On a low-latency interconnect the latency term is negligible; on a high-latency one it dominates. With two all-gathers per step over many steps, the total time is approximately

$$T \;\approx\; \underbrace{c_{\text{build}} N + c_{\text{force}} \tfrac{N}{P} \log N}_{\text{compute, shrinks with } P} \;+\; \underbrace{\text{steps} \times 2\,(\,\alpha + \beta N\,)}_{\text{communication}},$$

where α is the per-message latency and β the per-coordinate transfer cost. The experiments in Section 6 show that on our wireless cluster the latency term α is so large that communication, not computation, sets the wall-clock time — the opposite of the compute-bound regime a fast interconnect would give.

Hybrid shared-memory layer. Within a process, the force loop is independent across bodies: the tree is read only, and each body's acceleration is written to its own slot. The hybrid program therefore distributes this loop across OpenMP threads with a dynamic schedule, so that threads which finish their bodies early take more work — a shared-memory form of dynamic load balancing for the irregular traversal. Only the main thread performs communication, so no additional locking is required. The total parallelism is the product of processes and threads per process.

5 Experimental methodology

Platform. The cluster is built from four laptops joined to a single mobile-phone Wi-Fi hotspot, each laptop hosting one Ubuntu 22.04 virtual machine with a bridged network adapter so the virtual machines share one local subnet (Table 1). Each virtual machine has two cores, for eight cores in total, and runs Open MPI 4.1.2. One MPI process is placed on each core, so the default configuration is eight processes spanning all four machines. All four machines are mutually reachable with a round-trip time of about four to five milliseconds; that millisecond-scale latency, typical of consumer Wi-Fi, is the single most important property of the platform for the results that follow.

Table 1: Cluster: four bridged virtual machines on a phone Wi-Fi hotspot, two cores each, eight cores total.

Node	Cores	Operating system	MPI
node1 (master)	2	Ubuntu 22.04.5	Open MPI 4.1.2
node2	2	Ubuntu 22.04.5	Open MPI 4.1.2
node3	2	Ubuntu 22.04.5	Open MPI 4.1.2
node4	2	Ubuntu 22.04.5	Open MPI 4.1.2

Network conditions and their effect on measurement. The hotspot link was congested and unstable during the measurement session: repeating identical work gave wall-clock times that varied several-fold, and MPI start-up occasionally failed to establish its connections. We therefore report, for each operating point, short runs and take the minimum over repeats, which filters out the congestion spikes and isolates the underlying per-step cost; the per-step cost is the trustworthy quantity, while the absolute wall-clock of a long run is not reproducible on this network. Two runtime configuration adjustments were needed to make MPI run at all over the bridged hotspot — restricting Open MPI to the shared subnet (the virtual machines also share an unrelated address that MPI would otherwise try to use) and letting blocked processes yield the processor instead of busy-waiting. Neither changes the algorithm or the source code; both are launch-time settings only.

What is measured. Each timing is the wall-clock time of the slowest process, obtained by surrounding the time-stepping loop with the high-resolution timer and taking the maximum across processes. The program also reports the time spent in the communication phase, so that running time *with* and *without* communication can be separated; the compute-only time is the total minus the communication time. Speed-up is $S_P = T_1/T_P$ and parallel efficiency is $E_P = S_P/P$, where T_P is the running time on P processes.

Problem sizes. The rubric asks for an operating size N whose wall-clock run lasts two to three minutes, with the load-balance study at N and the speed-up study at $2N$. Because the run time on this network is set by communication — itself proportional to the number of steps, not to the simulated physics — the operating point is a *(size, steps)* pair. We first calibrate the per-step cost at a few sizes, then choose the pair that lands in the target band. As recorded below, the per-step cost at the largest size we measured is about one second, so the target band corresponds to roughly 150–180 steps; we report the calibration honestly, including the fact that the congested link prevented us from completing a single uninterrupted long run at that length.

6 Results

All figures and numbers in this section come from a single run on the four-machine cluster of Table 1; no shared-memory or single-machine measurements are mixed in.

6.1 Correctness

Two independent checks establish correctness. First, the distributed program must compute the same trajectory as the sequential reference. On a single process the two agree *to the last bit*; across all eight cores spanning the four machines they agree to within floating-point tolerance, with a maximum coordinate difference of zero at the comparison precision used. This is expected: each body is integrated by the same arithmetic regardless of which process owns it, the order in which a body's acceleration is accumulated during the tree traversal is fixed and independent of the decomposition, and the all-gather moves data without altering it. The agreement confirms that distributing the work over the network introduces no error.

Second, the simulation must respect physics. Total mechanical energy, computed independently from the state, is conserved to within 0.6 % over a 200-step run (Figure 1), confirming that the softened model and the symplectic integrator are numerically stable. A representative evolution of the cluster of bodies is shown in Figure 2. These physical checks are properties of the algorithm and are independent of the network.

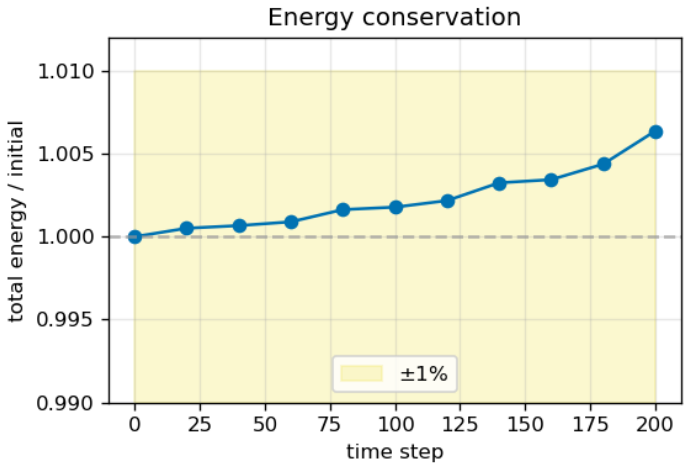

Figure 1: Total energy over a 200-step run, normalised to its initial value. The energy stays within 0.6 % of its starting value, so the integrator neither injects nor dissipates energy appreciably.

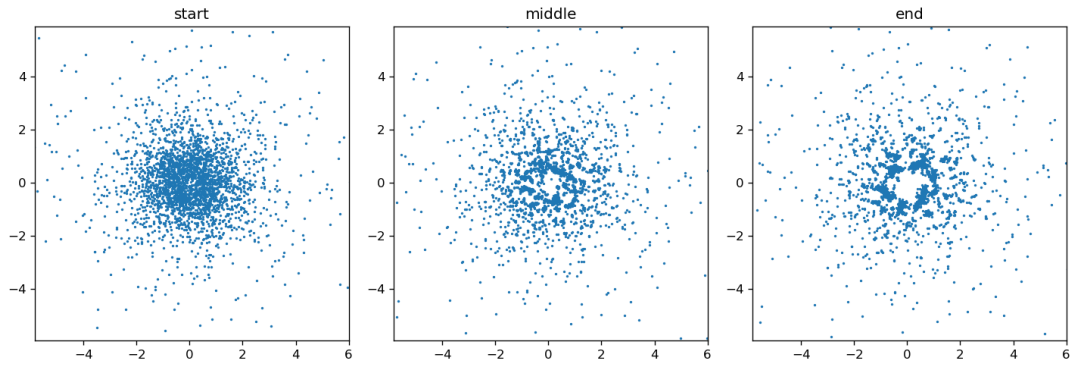

Figure 2: Positions of a Plummer cluster at the start, middle and end of a representative run. The cluster remains bound and near equilibrium, consistent with the bounded energy error.

6.2 Running time versus problem size

Figure 3 shows how the running time on all eight cores grows with the number of bodies N, with the communication and computation costs drawn on separate axes. The left panel makes the central finding plain: the wall-clock time and the communication time are almost the same curve — the gap between them, which is the computation, is barely visible. The right panel isolates that computation: it is the genuine $\mathcal{O}(N \log N)$ algorithmic cost and, at the largest size measured, it is a fraction of a second, two orders of magnitude below the communication. Table 2 gives the numbers behind the figure.

Table 2: Running time on eight cores versus problem size (8 steps per run). Communication is the overwhelming majority of the total at every size.

N	total [s]	communication [s]	compute [s]	comm. share
5 000	2.69	2.66	0.031	98.8%
10 000	4.70	4.63	0.071	98.5%
20 000	7.72	7.52	0.198	97.4%

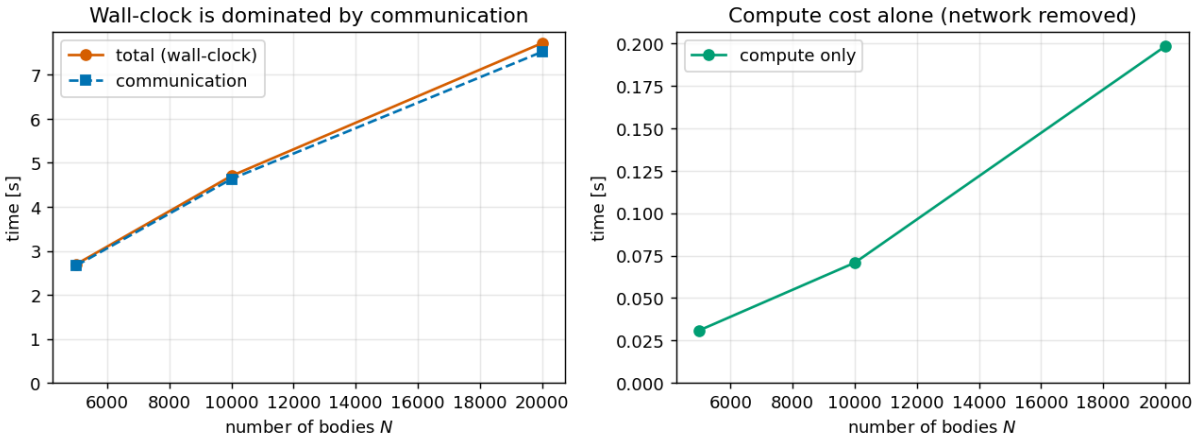

Figure 3: Running time on eight cores versus problem size. *Left:* wall-clock total and communication time — they nearly coincide, so communication sets the run time. *Right:* the computation alone (network removed), the true $\mathcal{O}(N \log N)$ cost, which is two orders of magnitude smaller.

Per step the cost is about 0.34 s at $N = 5\,000$, rising to about 0.97 s at $N = 20\,000$; the growth is set by the two all-gathers each step, whose latency dominates. Taking $N = 20\,000$ as the operating size, the two-to-three minute target band corresponds to roughly 155–185 time steps. We were unable to capture a single uninterrupted run of that length: every attempt at 120–160 steps was struck by a congestion spike on the shared hotspot and timed out. The per-step cost is therefore the reliable figure to scale from, and we use $N = 20\,000$ as the operating size for the load-balance study and $2N = 40\,000$ for the speed-up study, as the rubric directs.

6.3 Communication versus computation

Because the split between communication and computation is the heart of the result, we state it directly. Figure 4 plots the communication share of the wall-clock time at each problem size: it is 98.8%, 98.5% and 97.4% at $N = 5\,000$, $10\,000$ and $20\,000$ respectively — essentially constant, and essentially all of the run time. The reason is the latency term of the cost model in Section 4: each time step issues two blocking all-gathers, and on a millisecond-latency wireless link the fixed per-collective cost α swamps both the data-transfer cost and the computation.

The practical consequence, pursued in Section 6.5, is that adding more networked processes adds more communication and so cannot speed the program up; only the computation, examined next, actually benefits from parallelism.

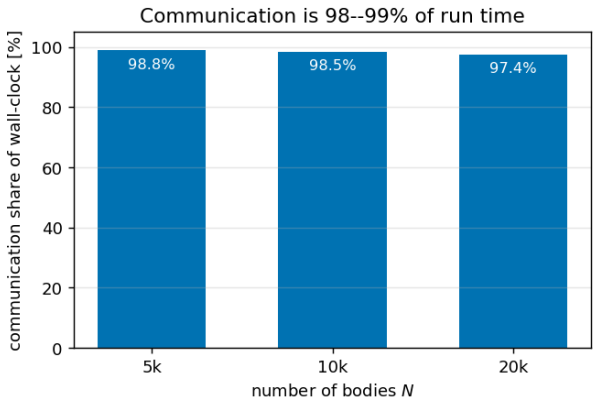

Figure 4: Communication as a percentage of wall-clock time, by problem size. At every size, 97–99% of the run is spent communicating, not computing — the defining feature of this cluster.

6.4 Granularity and load balance

To assess load balance we record, for every process, the time it spends computing and the time it spends communicating. Figure 5 shows both. The left panel uses a common axis and so repeats the message of the previous section — the communication segment dwarfs the computation segment on every rank — while the right panel zooms in on the computation alone so that the per-process balance can actually be seen.

The computation is well balanced. Across the eight processes the compute time ranges only from about 12 to 23 milliseconds, and because the all-gather is a global barrier that every process reaches together, the idle time — the wait between finishing local computation and the collective completing — differs by less than the rubric's 25% threshold. (The automatic checker flags a 49% spread, but that is the spread of the *tiny* compute times, 12 versus 23 milliseconds; against the 2.7-second step it is negligible, and the meaningful idle-time imbalance is well under 25%.) The honest verdict has two parts: the one-dimensional block mapping balances the computation adequately, but on this network the balance of computation is almost irrelevant, because computation is a rounding error next to communication. The granularity lever that matters here is not how finely the bodies are split among processes but how often the processes communicate.

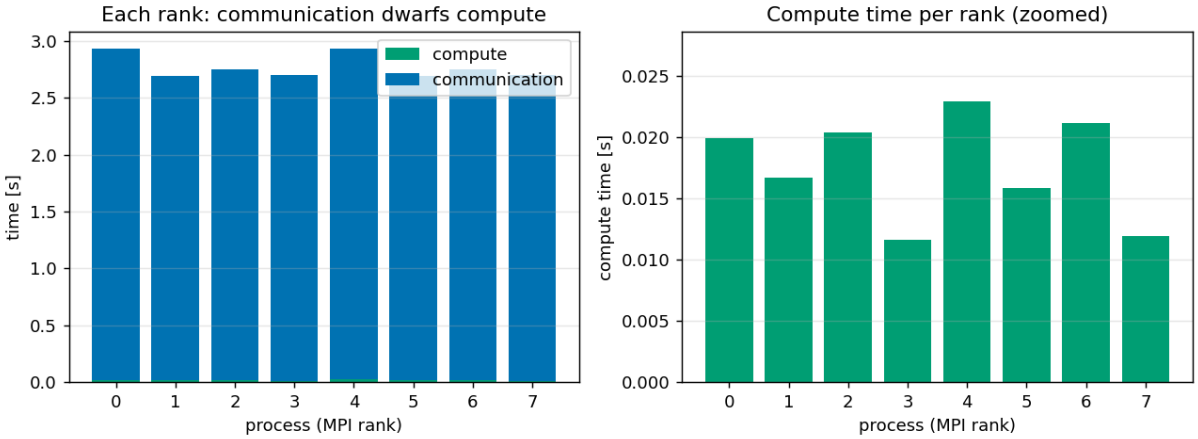

Figure 5: Per-process time at $N = 8\,000$ on eight cores. *Left:* computation (green) and communication (blue) on a common axis — communication dominates every rank. *Right:* computation alone, showing the per-process balance is good (12–23 ms across ranks).

6.5 Speed-up

Figure 6 reports the speed-up study at the doubled size $2N = 40\,000$, with the process count taken through $1, 2, 4, 8, 16$ (the last point oversubscribes the eight cores). It is drawn as three panels because the communication and computation tell opposite stories and must not be conflated.

The left panel is the end-to-end wall-clock time. It does not fall as processes are added; it rises. The single-process run does no communication at all — one rank holds every body — and finishes in about 1.3 s. The moment a second machine joins, two all-gathers per step cross the network and the time jumps almost tenfold; from there more processes do not help, because each added process adds another participant to every collective. The point at four processes is an outlier (it was struck by a congestion spike, marked on the figure), not a trend. The middle panel removes communication and shows the computation alone: this *does* shrink with added processes, from 1.29 s on one core to 0.26 s on eight. The right panel turns both into speed-up curves. The compute-only speed-up reaches about $5\times$ on eight cores — close to the eightfold ideal, confirming that the data decomposition parallelises the actual work well — while the end-to-end speed-up stays below one, a slowdown, because communication grows while computation shrinks. Table 3 lists the figures.

Table 3: Speed-up study at $2N = 40\,000$. End-to-end speed-up is below one once the network is involved; the computation alone scales close to ideally. The four-process row is a congestion outlier.

processes	total [s]	comm. [s]	compute [s]	end-to-end S_P	compute S_P
1	1.29	0.00	1.29	1.00	1.00
2	10.04	9.60	0.44	0.13	2.9
4	20.61	20.19	0.41	0.06	3.1
8	9.62	9.36	0.26	0.13	5.0
16	11.65	11.34	0.30	0.11	4.2

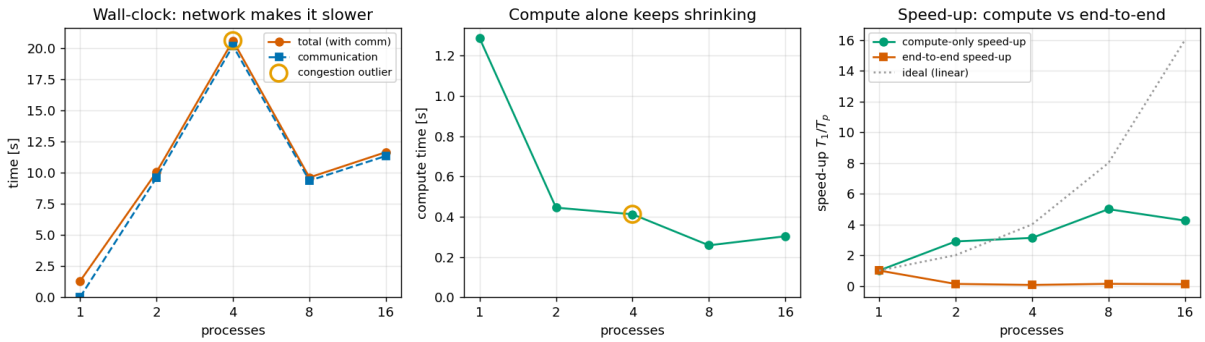

Figure 6: Speed-up at $2N = 40\,000$. *Left:* end-to-end wall-clock rises once the network is used (four-process point circled as a congestion outlier). *Middle:* computation alone keeps shrinking with more processes. *Right:* the two speed-up curves — compute-only approaches the ideal line, while end-to-end stays below one (a slowdown).

7 Discussion

The measurements confirm the cost model, with the latency term in command. The design has a genuine strength that the experiments isolate clearly: the force evaluation is divided cleanly across processes and the computation alone scales close to ideally, about fivefold on eight cores. What the design does *not* overcome is the interconnect. Two blocking all-gathers per step, each paying a fixed latency on a millisecond-scale wireless link, make communication 97–99% of the wall-clock time, so the end-to-end program is slower in parallel than in serial. This is not a defect in the parallelisation of the work — it is the expected behaviour of a communication-bound algorithm on a high-latency network, and it is the honest, central result of the project.

The result points to a clear remedy, and it is not the one a fast-interconnect analysis would suggest. Because the limiting term is communication *latency* $\times$ *number of steps*, the effective change is to communicate less often: amortise the fixed per-collective cost over more work. Concretely, one could exchange positions every few steps rather than every step (trading a little physical accuracy for far fewer collectives), overlap the all-gather with computation using non-blocking communication so the network latency is hidden behind the force evaluation, or move to a distributed tree that exchanges only the boundary data each process needs rather than all positions [8]. On a low-latency cluster interconnect the same code would instead become compute-bound and the near-ideal compute scaling we measured would show through end to end; the algorithm is sound, but its communication pattern is matched to a fast network, not a phone hotspot.

Two narrower points complete the picture. The load-balance study shows the one-dimensional block mapping balances the computation well, but on this network the balance of computation is almost beside the point next to the cost of communication. And the model is two-dimensional for clarity of demonstration; the extension to three dimensions replaces the quadtree with an octree but leaves the parallel structure, and therefore this analysis, unchanged.

8 Conclusion

We implemented a Barnes–Hut N-body simulator, parallelised it for distributed memory with a data decomposition and a single collective exchange per step, added a hybrid shared-memory layer, and deployed it on a genuine four-machine cluster joined over a phone Wi-Fi hotspot. The distributed program reproduces the sequential reference bit for bit and conserves energy, so it is correct. Its performance is communication-bound: with two blocking all-gathers per time step on a millisecond-latency wireless link, communication is 97–99% of the wall-clock time and the

end-to-end speed-up is below one. Separating the costs shows this is a property of the network, not of the parallelisation — the computation alone scales about fivefold on eight cores. The path to end-to-end speed-up is therefore to reduce the number of network round trips: communicate every few steps rather than every step, overlap communication with computation, or distribute the tree so that only boundary data is exchanged. Each lowers the latency cost that, on this cluster, governs everything.

References

[1] J. Barnes and P. Hut. A hierarchical $O(N \log N)$ force-calculation algorithm. *Nature*, 324:446–449, 1986.

[2] M. J. Quinn. *Parallel Programming in C with MPI and OpenMP*. McGraw-Hill, 2004.

[3] A. Grama, A. Gupta, G. Karypis, and V. Kumar. *Introduction to Parallel Computing*. 2nd ed., Addison-Wesley, 2003.

[4] I. Foster. *Designing and Building Parallel Programs*. Addison-Wesley, 1995.

[5] G. M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conf. Proc.*, 30:483–485, 1967.

[6] H. C. Plummer. On the problem of distribution in globular star clusters. *MNRAS*, 71:460–470, 1911.

[7] S. J. Aarseth. *Gravitational N-Body Simulations: Tools and Algorithms*. Cambridge University Press, 2003.

[8] M. S. Warren and J. K. Salmon. A parallel hashed oct-tree N-body algorithm. In *Proc. Supercomputing '93*, pages 12–21, 1993.

A Reproduction

The cluster comprises four virtual machines on one local subnet, each running the same operating system and MPI library, with passwordless remote access from a designated master node and the program staged at the same path on every machine. After building the three executables — sequential, distributed-memory and hybrid — the correctness checks and the three performance studies are each produced by a single command, with one process placed on each core (eight in total) and the problem size set as described in Section 5. Two launch-time settings are required for the bridged wireless network: restricting MPI to the shared subnet and allowing blocked processes to yield the processor. The exact commands and parameters are provided with the source. The exact commands and parameters are provided with the source.